



School of Artificial Intelligence
Jilin University

第九章 NP问题

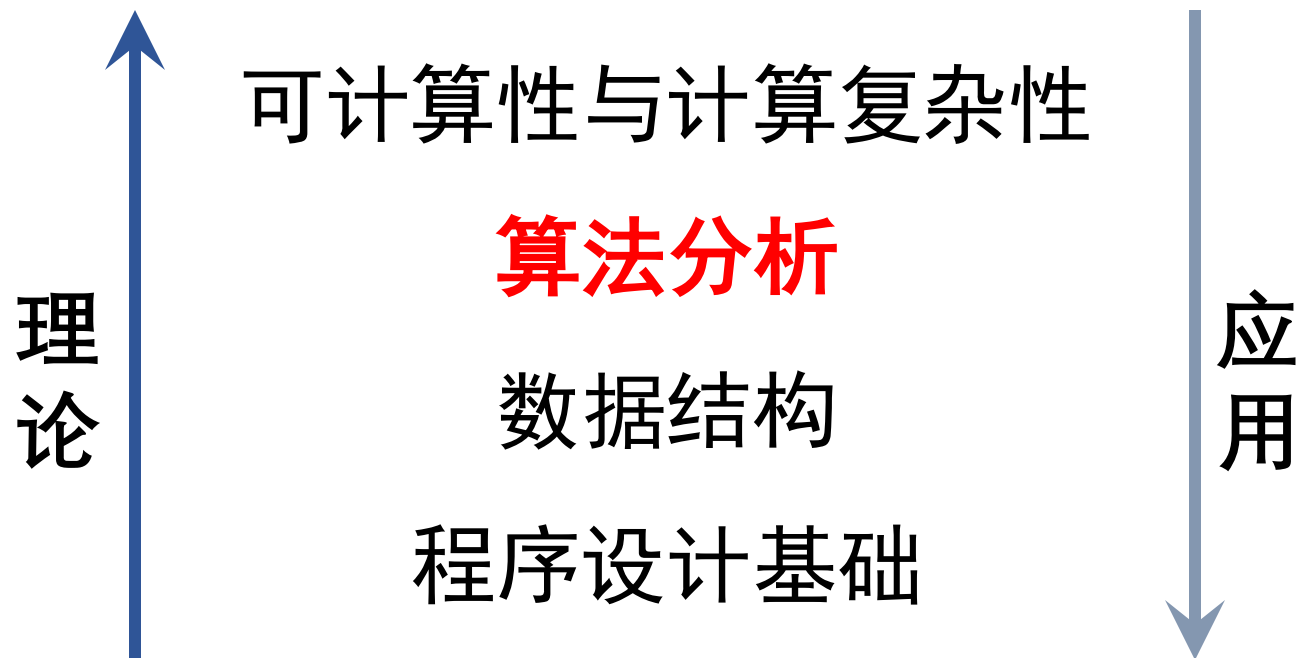
吉林大学人工智能学院

王元宏

- 9.1 图灵机与可计算性理论
- 9.2 计算复杂性理论
- 9.3 不确定性图灵机和不确定算法
- 9.4 NP问题和NP完全性
- 9.5 小结

- 9.1 图灵机与可计算性理论
- 9.2 计算复杂性理论
- 9.3 不确定性图灵机和不确定算法
- 9.4 NP问题和NP完全性
- 9.5 小结

为什么研究算法？



因为是必修课？

- 受过良好训练的计算机科学家知道怎样处理算法：如何构造算法、操作算法、理解算法、分析算法
- 这些知识远不止为了写出良好的计算机程序而准备的。算法是一种一般性的职能工具，它必定有助于我们对其他学科的理解，不管是化学、语言学、音乐，还是其他学科

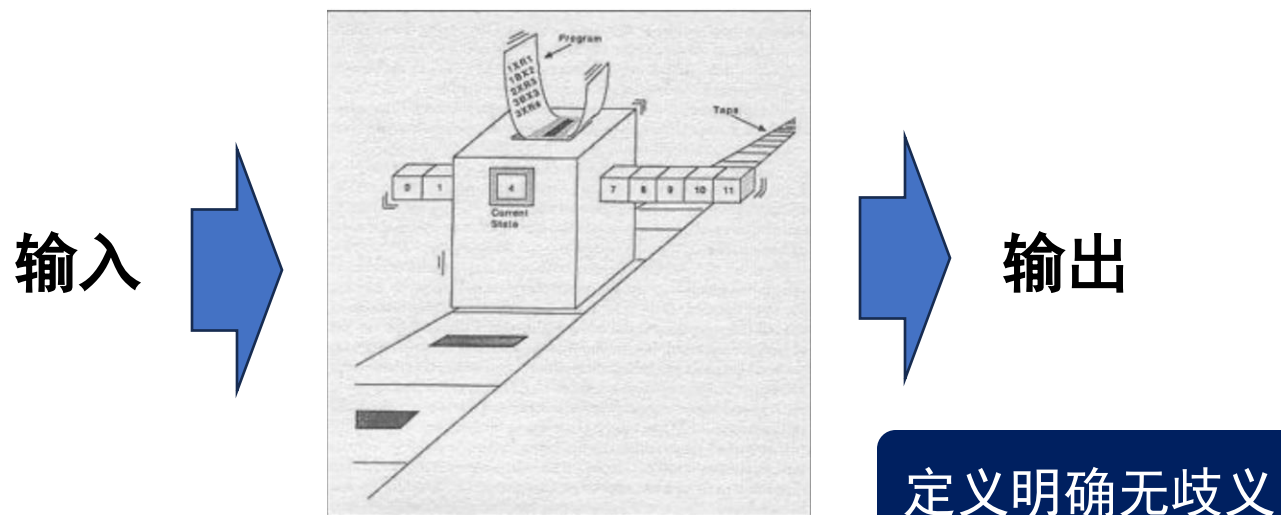
——Donald Knuth

- 算法不只是计算机科学的一个分支，它是计算机科学的核心。而且，可以毫不夸张地说，它与绝大多数科学、商业和技术都密切相关

——《Algorithmics: the spirit of computing》

因为大家都在研究？

什么是算法？



计算步骤有限

- 良定义的计算过程
- 一系列的**计算步骤**

计算机可实现的指令

算法即人类用笔和纸进行机械计算过程的抽象！

为什么研究算法？

为什么研究算法？

- 不想重复机械劳动！~~—(懒)—~~**——提高生产力**



为前线破译密码、计算炮弹轨迹的
人工计算员（human computer）



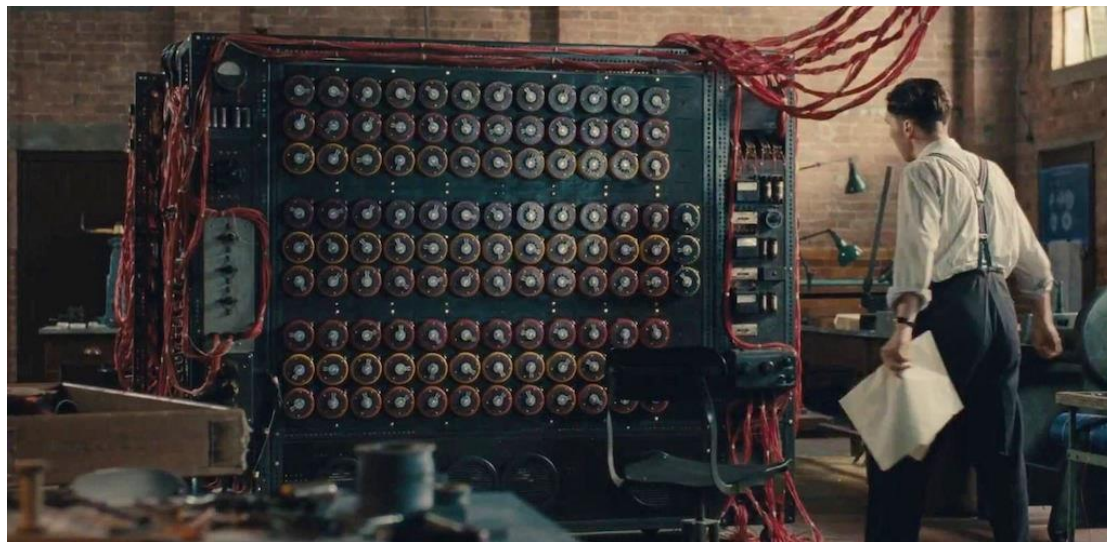
《三体》电视版中的人列计算机

为什么研究算法？

- 图灵创造图灵机破译恩格玛机



恩格玛机



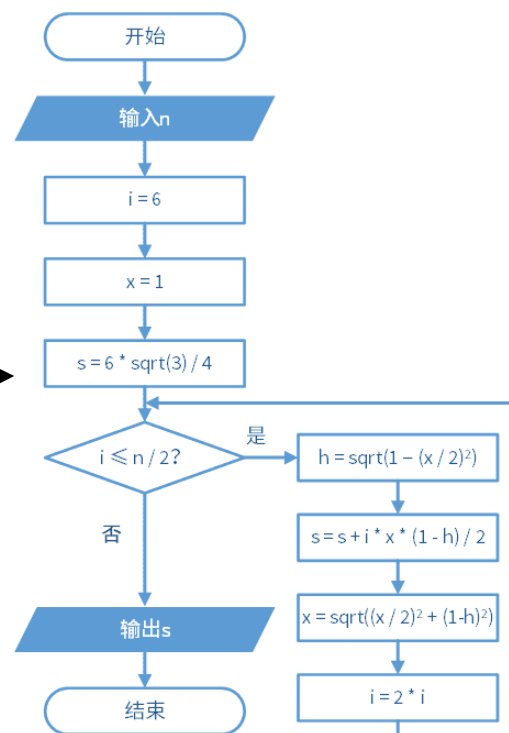
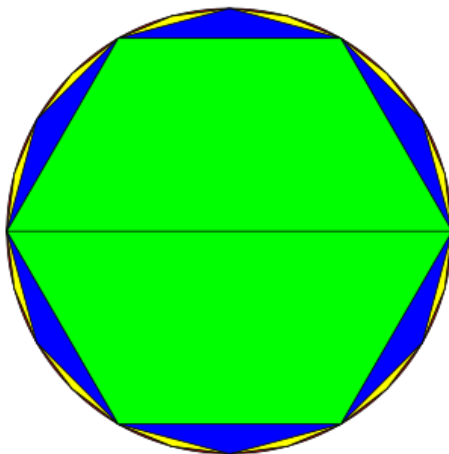
《模仿游戏》中的图灵机

为什么研究算法？

- 图灵创造图灵机破译恩格玛机
- 计算问题→算法→机器执行



刘徽
(约225年—约295年)
割圆术



- 不想重复机械劳动！ —(懒)— —— 提高生产力
- 从经验到形式化的认知跃迁 —— 成为科学
 - **不依赖特定对象的通用规则：**人类最早的计算行为（如结绳记事、筹算、珠算）本质上是机械性操作的经验总结。例如，中国古代《九章算术》中的“开方术”“方程术”，或是古埃及的乘法倍乘法，都是将具体计算步骤提炼为可复现的流程
 - **可编码、可验证：**人类解决问题时，常依赖直觉或“心算技巧”（如高斯速算 $1+2+\dots+100$ 的故事），但这些技巧往往是个体化的、难以传递的。算法通过形式化描述（如自然语言、流程图、伪代码），将“大脑中的隐性步骤”转化为可编码、可验证的显性规则

对算法的研究本质上是对 “如何利用计算解决问题” 的科学探索

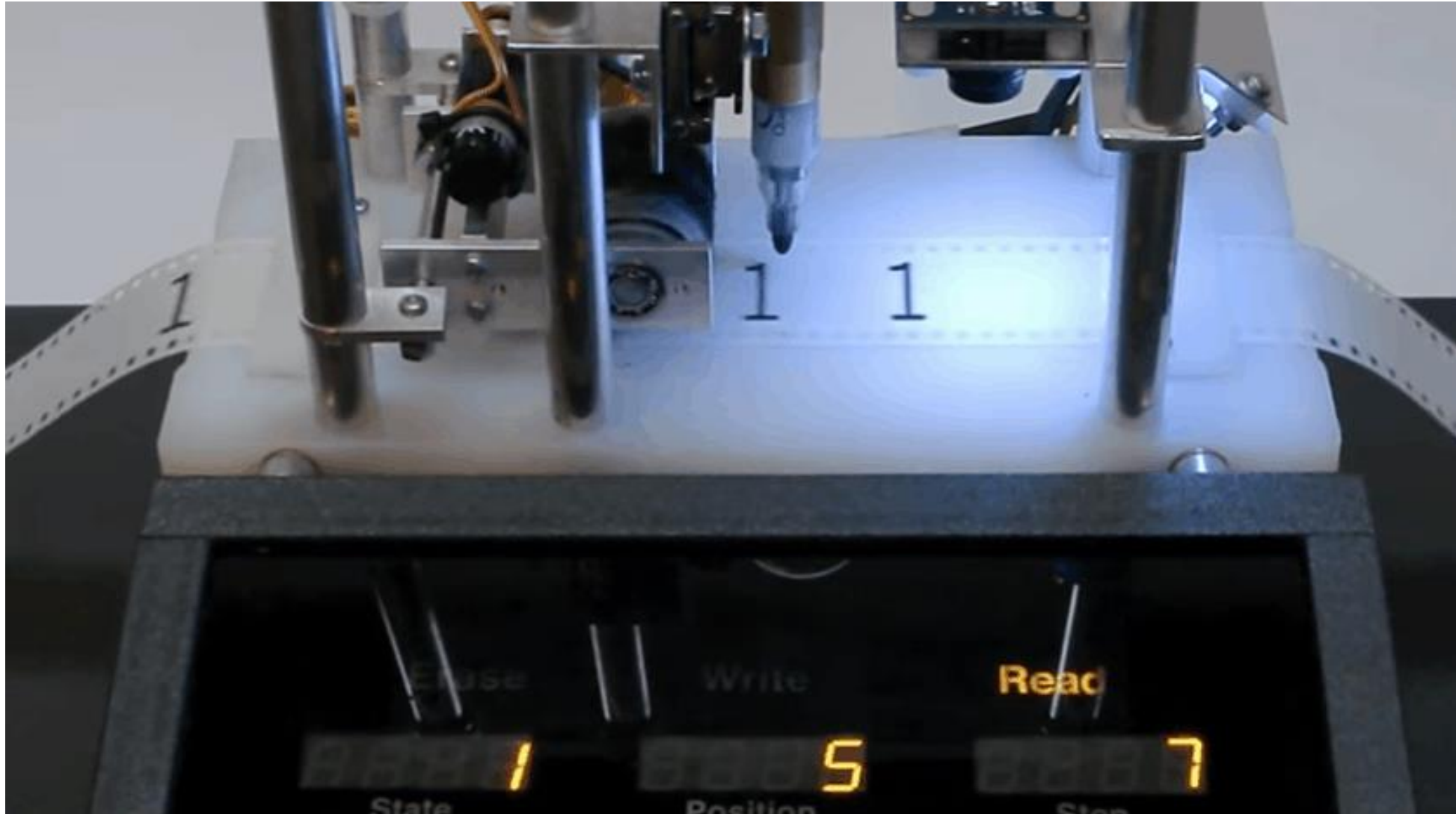
- 形式化定义明确
 - 如何进行计算
 - 哪些问题可以被计算
 - 计算的效率
- 使用数学语言——计算模型
 - 图灵机 (Turing machine)
 - λ 演算 (λ -calculus)
 - 递归函数论 (Recursive Function Theory)
 -

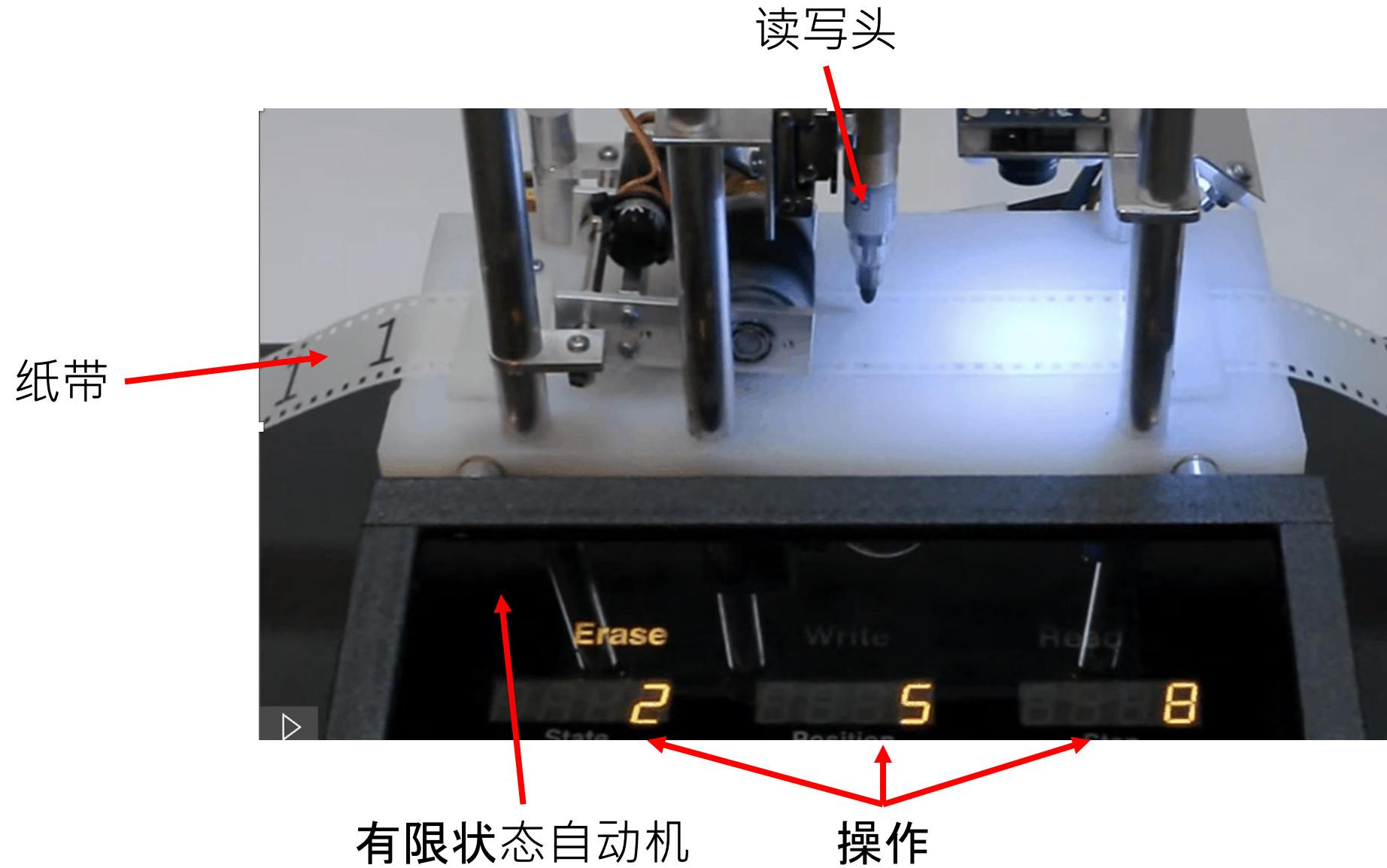
- 形式化定义明确
 - 如何进行计算
 - 哪些问题可以被计算
 - 计算的效率
- 使用数学语言——计算模型
 - **图灵机 (Turing machine)**
 - λ 演算 (λ -calculus)
 - 递归函数论 (Recursive Function Theory)
 -

- 常见基本计算操作
 - 数值运算：加减乘除、幂运算、取余等
 - 逻辑判断与条件分支
 - 循环与迭代控制
 - 数据存储与读取
- 复杂计算过程：由基本操作组成—— 算法

- 一条**无限长可读写**的一维纸带 —— 草稿纸
 - 用于存储数据
 - 记录中间结果
- 可读取、写入数据、可左右移动的读写头 —— 笔尖
 - 读取和修改纸带上的符号
- 有限状态自动机 —— 人类做机械计算任务时的大脑
 - 状态和状态之间的转移、及所执行的操作（流程图、规则表）
 - 初始状态和终止状态

当前状态	当前符号	写入符号	移动方向	下一状态
q0	+	1	右	q1

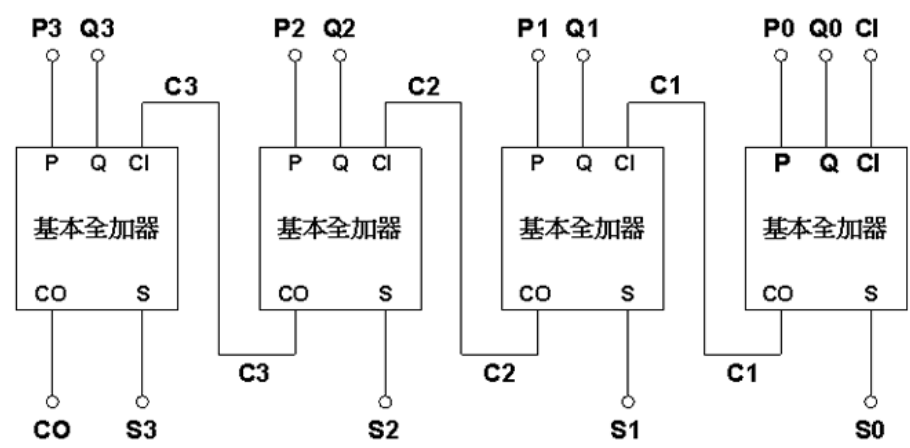
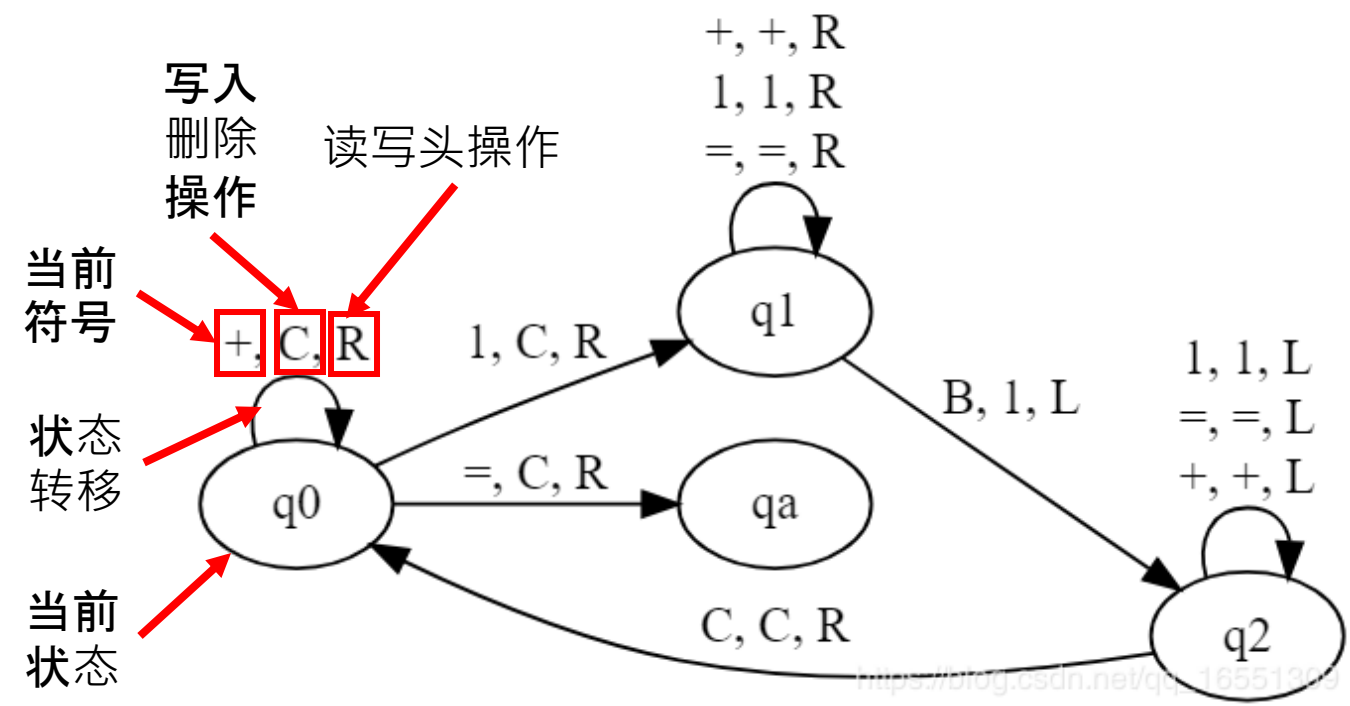




图灵机实现加法运算

- 使用一进制（任意进制皆可）编码数值
 - $3 \rightarrow 111, 2 \rightarrow 11$

计算 $3 + 2 = ?$ （任意两数加法都适用）



二进制加法器

图灵机可实现任意的机械计算！

- 哪些问题可以被计算？——即可以被图灵机计算
- 考虑一类特定的计算问题：可以形式化的判定问题
 - 输出是“是”或者“否”
- 例1：判断高次方程解的存在性，如 $ax^2 + bx + c = 0$
 - 丢番图方程是否有整数解： $a_1x_1^{b_1} + a_2x_2^{b_2} + \dots + a_nx_n^{b_n} = 0 \Rightarrow$ 第3个问题
 - 费马大定理： $x^n + y^n = z^n$ （当 $n > 2$ 时无整数解）
- 例2：判断一个数学命题是否为真
 - 素数是无穷的
 - 一阶逻辑公式是否是永真式 $\Rightarrow$ 第10个问题



David Hilbert
(1862-1943)
巴黎国际数学家会议ICM上
提出23个问题

We must know!
We will know!
——David Hilbert

- 图灵机停机问题  Hilbert的第10个问题
 - 判断一个图灵机 T 在输入 w 下是否会最终停下来（还是无限循环）
- 图灵在其硕士论文 “On Computable Numbers, With an Application to the *Entscheidungsproblem*” 中证明停机问题无法被机械计算！（图灵不可判定的）
- 即不存在一个图灵机，能够在有限步骤内判定停机问题

对于某些问题（即使是形式化的判断问题），不存在确定性、机械性的解决方法！

- 假设存在图灵机 H 可以解决停机问题
- 利用 H 构造新的图灵机 D
 - 输入任意一个图灵机的编码 $\langle P \rangle$
 - 用 H 判断 P 在输入 $\langle P \rangle$ 下是否停机
 - 若 H 输出 “是” (P 对自身停机), 则 D 进入无限循环
 - 如果 H 输出 “否” (P 对自身不停机), 则 D 停机
- 考虑将 D 的自身编码 $\langle D \rangle$ 输入 D
 - 若 H 判断 D 在输入 $\langle D \rangle$ 下停机 $\rightarrow D$ 无限循环
 - 若 H 判断 D 在输入 $\langle D \rangle$ 下不停机 $\rightarrow D$ 停机



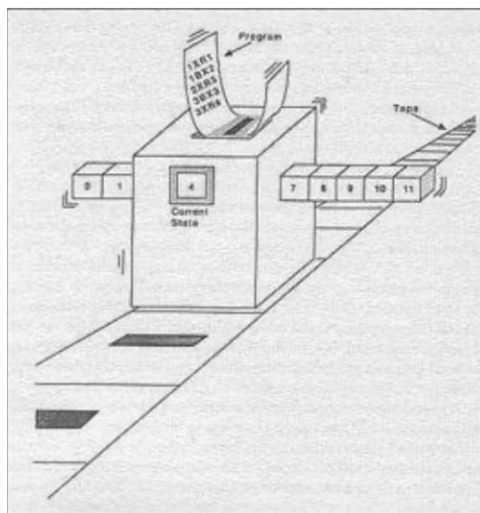
- 如何实现一个图灵机？
- 如果一个编程语言能够完全模拟图灵机，即称为图灵完全语言
- Brainfuck: <https://zh.wikipedia.org/wiki/Brainfuck>
- Python实现的Brainfuck解释器：
<https://github.com/BaseMax/BrainfuckInterpreterPython>

- 9.1 图灵机与可计算性理论
- 9.2 计算复杂性理论
- 9.3 不确定性图灵机和不确定算法
- 9.4 NP问题和NP完全性
- 9.5 小结

- 形式化定义明确
 - 如何进行计算
 - 哪些问题可以被计算
 - 计算的效率
- 考虑图灵机可计算的问题
 - 如何衡量问题的求解难易程度 —— 图灵机求解问题的步数

什么是算法？

输入



输出



定义明确无歧义

计算步骤有限

- 良定义的计算过程
- 一系列的**计算步骤**

图灵机可实现的指令

- 形式化定义明确
 - 如何进行计算
 - 哪些问题可以被计算
 - 计算的效率
- 考虑图灵机可计算的问题
 - 如何衡量问题的求解难易程度 —— 图灵机求解问题的步数
 - 也即算法的效率
 - 考虑可求解该问题的**最好**算法

思考：如何衡量算法的好坏？

- 运行时间是评价算法好坏的重要标准
 - 解决问题规模 n 需要的计算时间
- 下面比较三个算法的问题规模和计算时间之间的关系
 - 快速排序算法 $O(n \log n)$: 排序问题
 - Dijkstra算法 $O(n^2 \log n)$: 图的单源最短路径问题
 - 货郎担问题的动态规划法 $O(n^2 2^n)$: 求货郎担问题

- 假设用一台每秒 10^9 次的超大型计算机来计算

算法名称	时间复杂度	问题规模	运算量	计算时间(/ 10^9)
快速排序算法	$O(n \log n)$	10^5 个数据	$10^5 \times \log_2 10^5 \approx 1.7 \times 10^6$	1.7×10^{-3} 秒
Dijkstra算法	$O(n^2 \log n)$	10^4 个顶点	$(10^4)^2 \times \log_2 10^4 \approx 10^8$	0.1秒
货郎担问题的 动态规划算法	$O(n^2 2^n)$	10^2 个顶点	$2^{100} \approx 1.2 \times 10^{30}$	1.2×10^{21} 秒 $\approx 3.9 \times 10^{13}$ 年

- 多项式时间算法
 - $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3)$
- 非多项式时间算法(指数)
 - $O(2^n) < O(n!) < O(n^n)$
- 多项式时间算法是好算法！

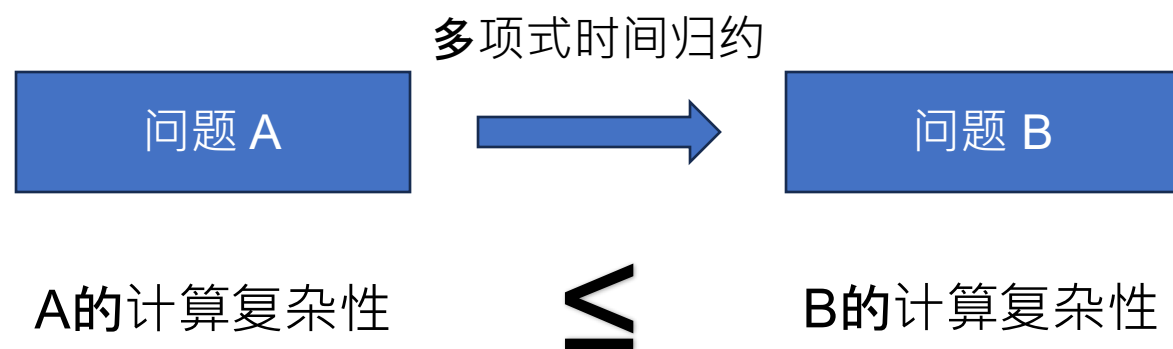
- 算法的复杂性
 - 是指解决问题的一个具体的算法的执行时间（即图灵机运行步骤的次数），是算法的性质
- 问题的计算复杂性
 - 是指这个问题本身的复杂程度，是问题的性质
- 例如排序问题。
 - 排序问题的复杂性是 $O(n \log n)$
 - 冒泡排序法是 $O(n^2)$ ，快速排序平均情况下是 $O(n \log n)$
 - 排序问题的复杂性是指在所有的解决该问题的算法中最好算法的复杂性

计算复杂性理论就是研究各种可计算问题的计算复杂性

- 多项式级的问题（即P问题）：
 - 这类问题已经设计出多项式级算法
 - 如排序、最小生成树、单源最短路径等
- 指数级的问题：
 - 这类问题已经设计出指数级算法，并且已证明不存在多项式级算法
 - 如带幂运算的正则表达式的全体性
- 尚不确定多项式级的问题：
 - 这类问题已经设计出指数级算法、但不能证明不存在多项式级算法
 - 如图着色问题、货郎担问题、0/1背包问题等

很多组合问题属于这种情况

- 如何判断问题的计算复杂性为多项式时间？
- 方法1：寻找问题的多项式时间算法
 - 找不到多项式时间算法是否意味着计算复杂性为指数时间？
 - 如何证明计算复杂性的下界（问题的难解性）？
- 方法2：多项式时间归约



- 不同的归约方法
 - 图灵归约：可以通过多次调用问题B的算法求解问题A
 - 多一归约：将A的输入转化成B的一个输入，B的输出就是A的结果

例：判定问题的多项式时间归约

- 考虑一类特定的计算问题：可以形式化的判定问题
 - 输出是“是”或者“否”

图着色问题

已知图 $G=\langle V, E \rangle$ ，求对 G 的 n 个顶点进行着色的一种方法（相邻顶点颜色不同），问最少需要多少种颜色？

图着色判定问题

已知图 $G=\langle V, E \rangle$ ，问图 G 是否可用 m 种颜色进行着色？

例：判定问题的多项式时间归约

最大团问题

图 $G=(V,E)$ 的一个完全子图叫做 G 的一个团(clique)。团的大小用所含的结点数表示。

求 G 内**最大**团和它的大小。

最大团判定问题

已知图 $G=<V,E>$ ，问图 G **是否**存在一个大小为 k 的团？

0/1背包问题

已知 n 个物品的效益值 p_i 和重量 w_i ，每个物品只能整取。在背包容量 M 的限定下如何选择，使选中物品的效益值之和**最大**？

0/1背包判定问题

在 M 限定下，对于给定效益值 X ，**是否**存在一组选择策略，使选中物品的效益值之和大于等于 X ？

最大团问题与其判定问题可以在多项式时间内相互归约

- 判定问题 $\rightarrow$ 优化问题
 - 设 $DCLIQUE(G, k)$ 为判定算法， n 是图 G 中点的个数，检验 $k=n, n-1, n-2, \dots$ 直到 $DCLIQUE(G, k)$ 输出为 1，则找到 G 的最大团大小
- 优化问题 $\rightarrow$ 判定问题
 - 求解图 G 的最大团问题，找到 G 的最大团大小 j ，如果 $j < k$ ，则 $DCLIQUE(G, k)$ 返回否，否则返回是

研究一个问题及其判定问题的计算复杂性在多项式时间级是等价的

计算复杂性研究本质上就是研究计算问题之间的归约关系

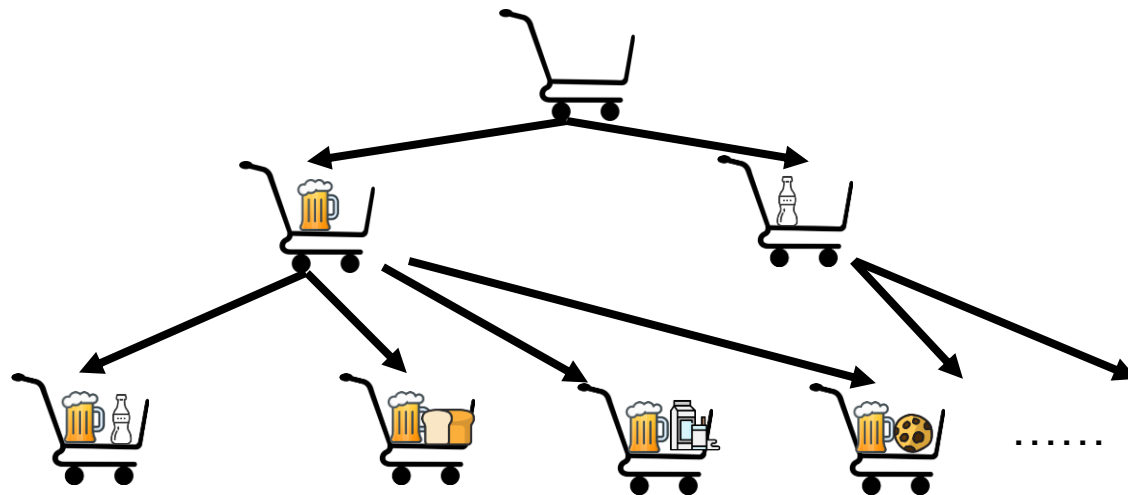
- 9.1 图灵机与可计算性理论
- 9.2 计算复杂性理论
- 9.3 不确定性图灵机和不确定算法
- 9.4 NP问题和NP完全性
- 9.5 小结

- 多项式级的问题（即P问题）：
 - 这类问题已经设计出多项式级算法
 - 如排序、最小生成树、单源最短路径等
- 指数级的问题：
 - 这类问题已经设计出指数级算法，并且已证明不存在多项式级算法
 - 如带幂运算的正则表达式的全体性
- 尚不确定多项式级的问题：
 - 这类问题已经设计出指数级算法、但不能证明不存在多项式级算法
 - 如图着色问题、货郎担问题、0/1背包问题等

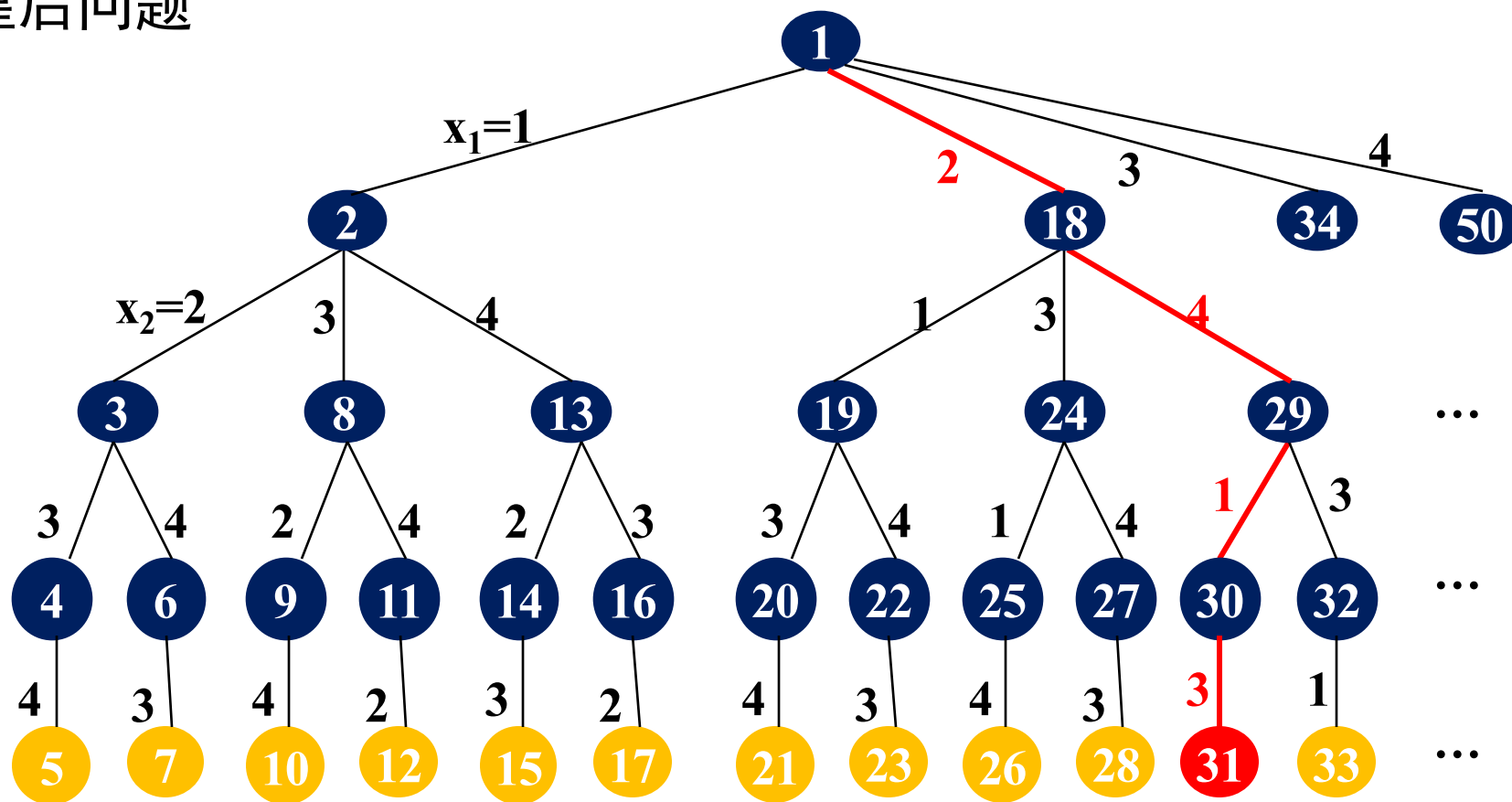
很多组合问题属于这种情况

思考：如何研究这些尚不确定多项式级问题的计算复杂性？

- 0/1背包问题



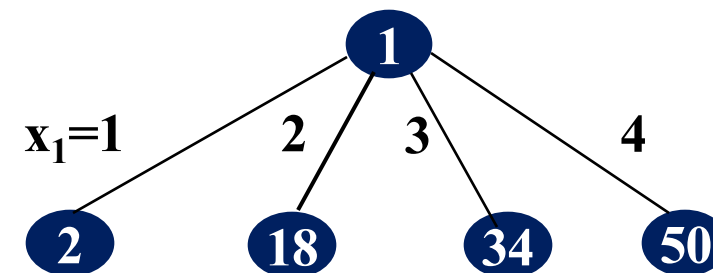
- 0/1背包问题
- 4-皇后问题



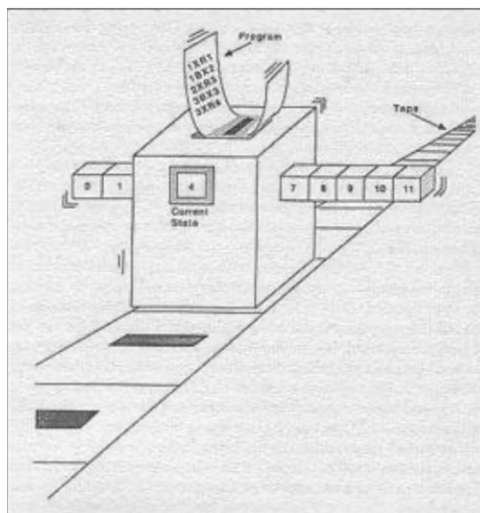
- 0/1背包问题
- 4-皇后问题
- 多步决策问题
- 最简单的求解方法：蛮力法
 - 每步尝试所有可能决策
 - 找到解（最优解）或者无解
- 用什么计算模型形式化该操作？
 - 从图灵机的角度看：每次状态转移都是**非确定**的（试探所有可能的转移）
 - 非确定性图灵机（non-deterministic Turing machine）

-
- 非确定性有限状态机 —— 人类做机械计算任务时的大脑
 - 状态和状态之间的转移、及所执行的操作（流程图、规则表）
 - 初始状态和终止状态

当前状态	当前符号	写入符号	移动方向	下一状态
q0	+	1	右	q1
q0	+	0	左	q2
q0	+	1	左	q1



输入



输出



定义明确无歧义

计算步骤有限

- 良定义的计算过程
- 一系列的**计算步骤**

非确定性图灵机可实现的指令

- 取消对运算“确定性”这一限制。即允许运算结果受限于某个特定的集合
- 包含这种不确定运算的算法称为不（非）确定算法
- 为描述不确定算法，引入三个非确定性图灵机支持的新语句，时间复杂度恒为 $O(1)$ ：
 - choice(S)：选取集合S中的一个元素
 - failure：发出不成功完成的信号
 - success：发出成功完成的信号
- 当存在一组选择发出成功完成的信息，算法成功地终止
- 否则，算法不成功地终止

- 不确定算法的设计步骤
 - 猜想阶段：也称为不确定阶段，基于不确定函数choice猜出一个解
 - 验证阶段：也称为确定阶段，用确定语句验证构造出的解是否是答案

- 问题描述：设 $A(1:n)$, $1 \leq i \leq n$, 是一个待排序的正整数集, 要求将其按非降次序排序

```
procedure NSORT(A, n)//对n个正整数排序
  B(1:n)←0 // 辅助数组
  for i←1 to n do // 猜想
    j←choice(1:n),
    if B(j) ≠0 then failure end
    B(j)←A(i)
  end
  for i←1 to n-1 do//验证B的次序
    if B(i) > B(i+1) then failure end
  end
  print(B)
  success
end
```

- 不确定的判定算法只需要回答“是” / “否”的问题，只产生0/1输出：
 - 0：当且仅当没有一种选择可导致success
 - 1：当且仅当至少存在一种选择导致success
- 为描述简洁，算法中不在显示输出语句

procedure DCK(G, n, k) // n 表示图 G 中点的个数

$S \leftarrow \emptyset$ // 求得的点集

for $i \leftarrow 1$ to k do

$t \leftarrow \text{choice}(1:n)$

if $t \in S$ then **failure** end

$S \leftarrow S \cup t$

end

for 使得 $i, j \in S$ 的每一对 (i, j) and $i \neq j$ do

if (i, j) 不是此图的边 then **failure** end

end

success

end

最大团判定问题

已知图 $G = \langle V, E \rangle$ ，问图 G 是否存在一个大小为 k 的团？

- 不确定算法的算法复杂度
 - 猜出一个解和验证一个解的时间复杂度
 - 即非确定性图灵机运行步骤的次数
- 排序问题的不确定算法NSORT的算法复杂度是多项式级
- 最大团判定问题的不确定算法DCK的算法复杂度是多项式级
 - 设图G由二进制邻接矩阵表示， n 是点个数， k 是判定值
 - 二进制转换输出参数集，其长度和为 $m = n^2 + \lfloor \log_2 k \rfloor + \lfloor \log_2 n \rfloor$
 - $O(n + k^2) = O(m)$

邻接矩阵

待判定的
结点数

结点数

- 9.1 图灵机与可计算性理论
- 9.2 计算复杂性理论
- 9.3 不确定性图灵机和不确定算法
- 9.4 NP问题和NP完全性
- 9.5 小结

- 回顾P问题
 - 所有可在多项式时间内用**确定**算法（确定性图灵机）求解的**判定问题**的集合
- NP问题
 - 所有可在多项式时间内用**不确定**算法（非确定性图灵机）求解的**判定问题**的集合
 - 即能够用不确定算法在多项式时间里猜出一个解和验证一个解
- 可以认为P问题是NP问题的一种特例，即choice(S)中，S仅有唯一一个元素，因此 **$P \subseteq NP$** 成立

几乎所有组合问题对应的判定问题都是NP问题

- 多项式级的问题（即P问题）：
 - 这类问题已经设计出多项式级算法
 - 如排序、最小生成树、单源最短路径等
- 指数级的问题：
 - 这类问题已经设计出指数级算法，并且已证明不存在多项式级算法
 - 如带幂运算的正则表达式的全体性
- 尚不确定多项式级的问题：
 - 这类问题已经设计出指数级算法、但不能证明不存在多项式级算法
 - 如图着色问题、货郎担问题、0/1背包问题等

很多组合问题属于这种情况

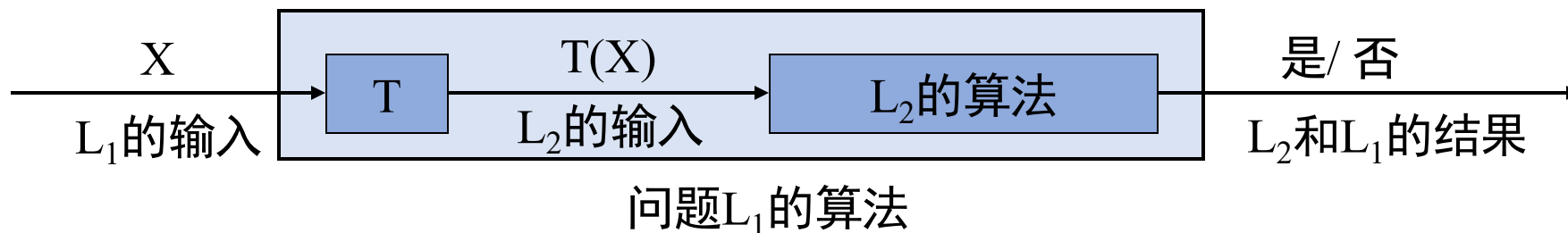
这些问题是否是多项式级的？ $P=NP$ ？ $P\neq NP$ ？

- 计算复杂性研究本质上就是研究计算问题之间的归约关系
- 找出NP问题中最难的问题 —— 所有NP问题都能归约到该问题
 - 是否存在?
- 研究该问题是否存在多项式时间确定算法或者是否可以归约到P问题

- NP问题 L_1 和 L_2 ， L_1 归约到 L_2 是指：

符号表示： $L_1 \leq_p L_2$

- 能够找到一个转换函数 $T(x)$ ：使得问题 L_1 的任意一个合法输入 x 转换成问题 L_2 的一个合法输入 $T(x)$
- 将 $T(x)$ 输入求解问题 L_2 的算法时，会得到问题 L_2 的一个输出
- 问题 L_1 对于输入 x 的输出“是”，当且仅当问题 L_2 对 $T(x)$ 的输出是“是”



思考：为什么不用图灵归约？

- 多项式时间归约
 - T是问题 L_1 归约到 L_2 的转换函数，如果计算T的算法是多项式级的，则称问题 L_1 可以多项式时间归约到 L_2 ，记为 $L_1 \leq_p L_2$
 - T称为归约函数，其算法称为归约算法
- 多项式时间归约具有传递性
 - 对于问题 L_1 、 L_2 、 L_3 ，如果 $L_1 \leq_p L_2$ ， $L_2 \leq_p L_3$ ，则 $L_1 \leq_p L_3$
- 当 L_1 可以多项式时间归约到 L_2 时
 - 如果 L_2 多项式级可解决， L_1 亦然
 - 如果 L_2 指数级可解决， L_1 亦然

- 找出NP问题中最难的问题 —— 所有NP问题都能归约到该问题
- 问题L是NP-完全的，或NPC的，当满足：
 - $L \in NP$
 - 对于每个 $L' \in NP$ ，有 $L' \leq_p L$
- 问题L是NP-难的，或NP-hard的，当满足：
 - 对于每个 $L' \in NP$ ，有 $L' \leq_p L$
- 比较NP-完全问题与NP-难问题
 - NP-hard比NPC范围广， NP-Hard同样难以找到多项式算法，有可能比所有NPC的时间复杂度更高从而更难以解决
 - NP-hard 不一定是NP问题；但NPC一定是NP问题

NP-完全问题是否存在？

第一个NP-完全问题——可满足性(SAT)问题

- 问题描述：对于任意给定的一个合取范式F, F是否可满足？
- 合取范式F：
 - 令 $x_1, x_2, \dots, x_n$ 表示布尔变量， $\neg x_i$ 表示 x_i 的非
 - 一个文字 σ 即可以是一个变量也可以是它的非
 - 合取符号 $\wedge$ 表示与关系；析取符号 $\vee$ 表示或关系
 - 公式形如 $A_1 \wedge A_2 \wedge \dots \wedge A_k$, A_j 是形如 $\vee \sigma$ 的子式，称为合取范式(CNF)
- F可满足
 - 存在一组赋值给 $x_1, x_2, \dots, x_n$ ，使F值为真

- 合取范式 $F_1 = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge \neg x_2$
 - 令 $(x_1, x_2, x_3) = (1, 0, 1)$, F_1 成真赋值, F_1 是可满足的.
- 合取范式 $F_2 = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge x_2 \wedge \neg x_3$
 - F_2 不存在真赋值, 不是可满足的

Procedure EVAL(F, n)

//判定命题 F 是否为可满足的。变量为 X_i , $1 \leq i \leq n$

for $i \leftarrow 1$ to n do//选取一组真值指派

$x_i \leftarrow \text{choice}(\text{true}, \text{false})$

end

if $F(x_1, \dots, x_n) = \text{true}$ then

success

else

failure

end

end

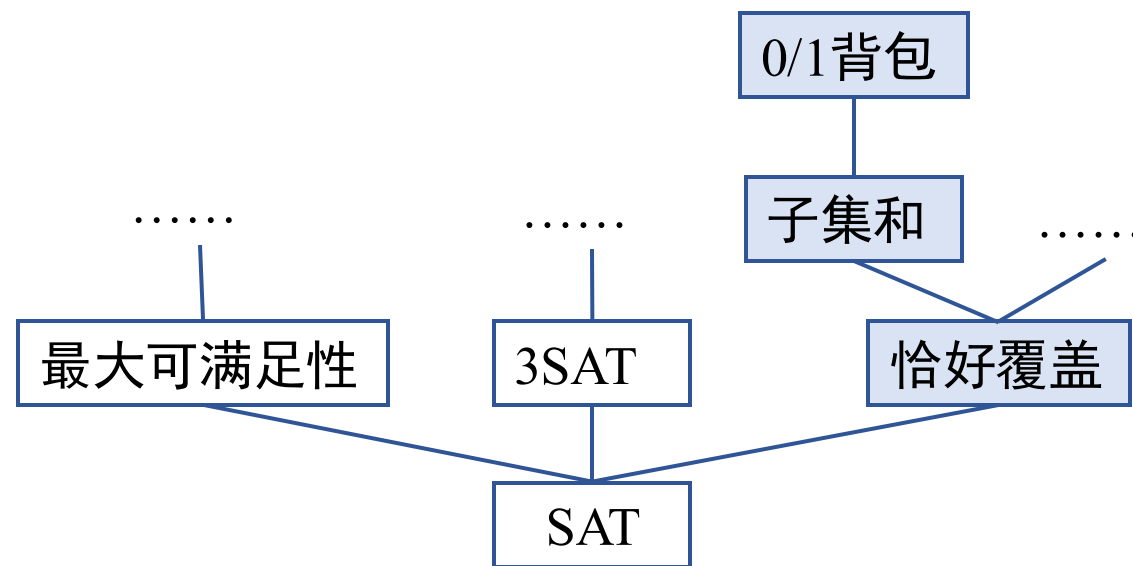
可满足性问题是一个NP问题

SAT问题是NP完全的

- 任何NP问题都可以在多项式时间归约为SAT问题
- SAT问题是第一个NPC问题
- 证明
 - 给定任意NP问题 L 和输入 I
 - 令 NTM 是多项式时间求解问题 L 的非确定性图灵机
 - 由 NTM 和 I 在多项式时间（由 L 的NP问题性质）内构造合取范式 F
 - 证明 L 判定 I 为“是”（即 NTM 接受 I ）当且仅当 F 是可满足的



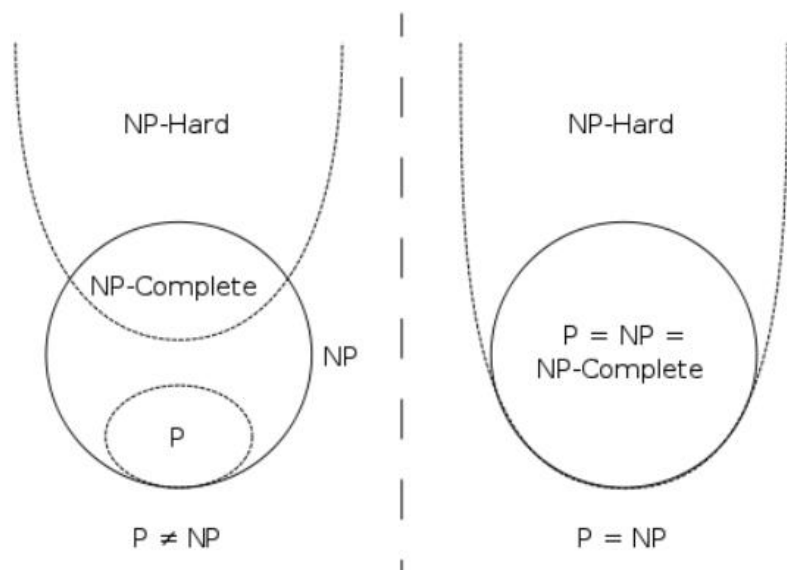
Stephen Cook
(1939年-)
NPC理论
图灵奖得主



现在NPC问题已超过3000个

[Karp的二十一个NP-完全问题](#)

- 计算复杂性研究本质上就是研究计算问题之间的归约关系
- 找出NP问题中最难的问题 —— 所有NP问题都能归约到该问题
 - 是否存在?
 - 存在! 现在NPC问题已超过3000个!
- 研究该问题是否存在多项式时间确定算法或者是否可以归约到P问题



We must know!
We will know?



- 一般认为 $NP \neq P$
- 因此，若一个问题是NPC的，那么一般认为不存在多项式时间的确定算法
 - 设计近似算法，如FRPAS, FPTAS
- 如何证明问题 $L \in NP$ 是NPC?
 - 基于一个已知 NPC-问题 L' , 证明 $L' \leq_p L$,
 - 而 “ $\leq_p$ ” 的关键在于能够从 L' 的实例 I' (在多项式时间内) 构建出 L 的一个实例 I

- 问题描述: 给定有穷集 $A = \{a_1, a_2, \dots, a_n\}$ 和 A 的子集的集合 $W = \{S_1, S_2, \dots, S_m\}$ 。
设 $U \subseteq W$, 如果 U 中的集合元素不相交且并集等于 A , 则称 U 是 A 的恰好覆盖。
问当前实例是否存在这样的 U ?
- 设 $A = \{1, 2, 3, 4, 5\}$, $W = \{S_1, S_2, S_3, S_4\}$, $S_1 = \{1, 2\}$, $S_2 = \{1, 3, 4\}$, $S_3 = \{2, 4\}$, $S_4 = \{2, 5\}$,
 - 则 $\{S_2, S_4\}$ 是 A 的恰好覆盖
 - 如果把 S_4 修改为 $S_4 = \{3, 5\}$, 则不存在 A 的恰好覆盖
- 恰好覆盖问题是 **NP-完全** 的

- 问题描述：给定正整数集合 $X=\{x_1, x_2, \dots, x_n\}$ 及正整数 N , 问存在 X 的子集 T , 使得 T 中的元素之和等于 N ?
- 已知子集和问题是 NP 问题, 求证子集和问题是 NP-完全的
- 证明思想: 往证恰好覆盖 $\leq_p$ 子集和
 - 给定有穷集 $A=\{a_1, a_2, \dots, a_n\}$ 和 A 的子集的集合 $W=\{S_1, S_2, \dots, S_m\}$
 - 对应的子集和形如 $X=\{x_1, x_2, \dots, x_m\}$ 及非负整数 N
 - 从 W 中选择 S_j 恰好覆盖 A 相当于从 X 中选择 x_j 等于 N

S_j 对应 x_j

- 每个 x_j 和 N 都可表示成 $n \times k$ 位的二进制数, $k = \lceil \log_2(m+1) \rceil$
- 将这 $n \times k$ 位分成 n 段, 每段 k 位二进制
- 构造二进制 N , 每段的最右位值为1
- 基于 S_j 构造二进制 x_j , 如果 $a_i \in S_j$, 第 i 段的最右位值为1, 否则全0

思考：为什么每段需要 k 位？

- 恰好覆盖问题原始实例:

- $A = \{a_1, a_2, a_3, a_4\} = \{2, 3, 5, 8\}$, $W = \{S_1, S_2, S_3\}$,
- $S_1 = \{a_1, a_2\} = \{2, 3\}$, $S_2 = \{a_1, a_3, a_4\} = \{2, 5, 8\}$, $S_3 = \{a_2\} = \{3\}$
- $S_2 \cup S_3$ 是 A 的全覆盖

- 转化为二进制的子集和问题

- $n=4$, $m=3$, 则 $k = \lceil \log_2(3+1) \rceil = 2$
- 定义函数 $f(i, j) = \begin{cases} 01 & a_i \in S_j, j=0, 1..m, 0 \text{表示集合} A \\ 00 & \text{否则} \end{cases}$

- $N = 01010101$

- $x_1 = 01010000$ 即 $f(1,0)f(2,0)f(3,0)f(4,0)$

- $x_2 = 01000101$

- $x_3 = 00010000$

易见 $x_2 + x_3 = N$

上个思考：为什么每段需要 k 位？
求子集和问题时，最多 m 个数相加，
 k 保证了不会进位到上一段中

思考：子集和 $\leq p$ 0/1 背包问题？

- 9.1 图灵机与可计算性理论
- 9.2 计算复杂性理论
- 9.3 不确定性图灵机和不确定算法
- 9.4 NP问题和NP完全性
- 9.5 小结

- 理论计算模型 —— 形式化机械计算
 - 确定性图灵机
 - 不确定性图灵机
- 图灵机停机问题：证明了并非所有问题都是可计算的
- 计算复杂性
 - 求解问题的最好算法的算法复杂性
 - 认为多项式时间的问题是可解的
 - 通过归约来研究问题的计算复杂性

- 不确定算法：不确定图灵机对应的算法，用于研究组合问题的计算复杂性
- P-问题：所有可在多项式时间内用**确定**算法求解的判定问题的集合
- NP-问题：所有可在多项式时间内用**不确定**算法求解的判定问题的集合
- NP-完全问题
 - 所有NP问题都可以多项式时间归约的NP问题
 - SAT问题是第一个NP-完全问题
 - NP-完全性的证明
- 若能证明任一NP-完全问题可以多项式时间归约到P问题（即发现多项式时间确定算法），则证明了 $NP=P$